📘 AI/ML Internship – Day 82 Notes & Tasks

🟢 Module 10: Project Requirements & SRS Documentation

📌 Part 1: Recap of Day 81

Yesterday we learned:

- ✅ What is a Capstone Project?
- ✅ Project Problem Identification
- ✅ Requirement Analysis
- ✅ System Design
- ✅ Technology Selection
- ✅ Project Architecture
- ✅ Project Planning

Today we will learn:

- What is SRS?
- Why is SRS important?
- Functional Requirements
- Non-Functional Requirements
- User Requirements
- System Requirements
- Project Scope
- Project Constraints
- Use Cases
- Creating an SRS document

📌 Part 2: What is SRS?

Definition

SRS stands for Software Requirements Specification.

It is a document that describes **what the software should do and what requirements it must satisfy**.

Simple Definition

SRS is basically a **complete written plan of what your application should contain and how it should behave**.

Before developers start coding, they should understand:

What are we building?

Why are we building it?

Who will use it?

What should it do?

📌 Part 3: Why is SRS Important?

Imagine four interns are working on the same project.

One person thinks:

"The system only needs login."

Another thinks:

"We also need admin login."

Another thinks:

"Users should be able to upload files."

Another thinks:

"We don't need file uploads."

This creates confusion.

An SRS document gives everyone the **same understanding of the project requirements**.

📌 Part 4: Main Sections of an SRS

A basic SRS can contain:

1. Introduction

2. Problem Statement
3. Objectives
4. Scope
5. Target Users
6. Functional Requirements
7. Non-Functional Requirements
8. Technology Requirements
9. System Requirements
10. User Requirements
11. Use Cases
12. Constraints
13. Assumptions
14. Expected Outcome

📌 Part 5: Introduction

The introduction gives a brief overview of the project.

Example

Project: AI Resume Screening System

The AI Resume Screening System is an intelligent application designed to analyze resumes and identify candidates based on required skills and qualifications.

📌 Part 6: Problem Statement

The problem statement explains **the real-world problem that the project is trying to solve**.

Example

Recruiters receive hundreds of resumes for a single job position. Manually reviewing every resume requires significant time and effort.

The proposed system uses AI and NLP techniques to automatically analyze resumes and assist recruiters in identifying suitable candidates.

📌 Part 7: Objectives

Objectives explain what you want to achieve.

Example

The objectives are:

- Automatically analyze resumes.
- Extract candidate skills.
- Compare skills with job requirements.
- Generate candidate scores.
- Reduce manual screening time.
- Help recruiters shortlist candidates.

📌 Part 8: Project Scope

Definition

Scope defines **what is included and what is not included in the project**.

Example

Included

- Resume upload
- Resume text extraction
- Skill extraction
- Candidate scoring
- Recruiter dashboard
- Report generation

Not Included

- Automatic hiring
- Salary negotiation
- Final recruitment decision

This prevents the project from becoming unnecessarily large.

📌 Part 9: Target Users

Identify who will use your application.

For example:

AI Resume Screening System

- Recruiters
- HR Managers
- Company Administrators

Student AI Assistant

- Students
- Teachers
- Administrators

📌 Part 10: Functional Requirements

Definition

Functional requirements describe **what the system should do**.

Think of them as the application's features.

Example

For a Resume Screening System:

1. User should be able to register.
2. User should be able to log in.
3. Recruiter should be able to upload resumes.
4. System should extract resume text.
5. System should identify candidate skills.
6. System should compare skills with job requirements.
7. System should calculate a candidate score.
8. System should display the results.
9. User should be able to download reports.

These are functional requirements.

📌 Part 11: Non-Functional Requirements

Definition

Non-functional requirements describe **how the system should perform**.

Examples:

Performance

The system should return results within a reasonable time.

Security

User information should be protected.

Usability

The application should be easy to use.

Reliability

The application should work consistently.

Scalability

The system should support an increasing number of users.

📌 Part 12: Functional vs Non-Functional Requirements

Functional	Non-Functional
What the system does	How the system performs
Login	Security
Upload PDF	Performance
Generate prediction	Reliability
Search documents	Scalability
Generate report	Usability

Easy way to remember:

Functional = Features

Non-Functional = Quality

📌 Part 13: User Requirements

User requirements describe what users expect from the application.

Example:

The recruiter should be able to upload a resume.

The recruiter should be able to view the candidate score.

The student should be able to ask questions to the AI chatbot.

📌 Part 14: System Requirements

System requirements describe the technical environment required to run the application.

Hardware

Example:

- Processor: Intel i5 or equivalent
- RAM: 8 GB or higher
- Storage: 10 GB available

Software

- Windows/Linux
- Python
- FastAPI
- Database
- Required AI libraries

The exact requirements depend on the project.

📌 Part 15: Use Cases

A **Use Case** describes how a user interacts with the system to achieve a goal.

Example

Use Case: Upload Resume

Recruiter

↓

Login

↓

Upload Resume

↓

System Reads Resume

↓

AI Analyzes Resume

↓

Skills Extracted

↓

Score Generated

↓

Result Displayed

📌 Part 16: Project Constraints

Constraints are limitations that may affect development.

Examples:

- Limited dataset
- Limited computing resources
- API usage limits
- Limited development time
- Limited budget
- Model performance limitations

📌 Part 17: Assumptions

Assumptions are conditions we expect to be true.

Example:

- Users have internet access.
- Uploaded resumes are readable.
- Required APIs are available.
- Training data is properly labeled.

📌 Part 18: Expected Outcome

This describes what the final application should achieve.

Example:

The system should analyze resumes automatically and provide recruiters with candidate scores and relevant skill information.

📌 Part 19: Example SRS Structure

Your project document can follow this structure:

1. Project Title

2. Introduction

3. Problem Statement

4. Objectives

5. Scope

6. Target Users

7. Functional Requirements

8. Non-Functional Requirements

9. Technology Stack

10. Hardware Requirements

11. Software Requirements

12. Use Cases

13. System Architecture

14. Constraints

15. Assumptions

16. Expected Outcome

17. Future Enhancements

📌 **Part 20: Why Should We Complete SRS Before Coding?**

Without SRS:

❌ Requirements may change frequently.

❌ Team members may misunderstand the project.

❌ Important features may be missed.

❌ Development may become disorganized.

With SRS:

✅ Clear project requirements.

✅ Better team coordination.

✅ Easier development.

✅ Easier testing.

✅ Easier project documentation.

📌 Part 21: Theory Questions

1. What is SRS?

SRS stands for Software Requirements Specification. It is a document that describes the requirements and expected behavior of a software system.

2. What are functional requirements?

Functional requirements describe the features and operations that the system should perform.

3. What are non-functional requirements?

Non-functional requirements describe qualities such as performance, security, usability, reliability, and scalability.

4. What is project scope?

Project scope defines the features and activities included and excluded from the project.

5. What is a use case?

A use case describes how a user interacts with a system to achieve a specific goal.

📌 Part 22: Interview Questions

What is the purpose of an SRS document?

An SRS provides a clear and detailed description of software requirements so that developers, testers, and stakeholders have a common understanding of the project.

Difference between functional and non-functional requirements?

Functional requirements describe **what the system does**, while non-functional requirements describe **how well the system performs**.

Why is project scope important?

It prevents unnecessary expansion of the project and clearly defines what features will be developed.

What is the difference between user requirements and system requirements?

User requirements describe what users expect from the system, while system requirements describe the technical and operational requirements needed to implement and run it.

🎯 Tasks for Day 82

🧠 Theory

1. Define SRS.
2. Explain why SRS is important.
3. Differentiate functional and non-functional requirements.
4. Explain project scope.
5. What is a use case?

💻 Practical Task 1

Prepare the following for your selected project:

- Project Title
- Introduction
- Problem Statement
- Objectives
- Target Users

💻 Practical Task 2

Write at least **10 functional requirements** for your project.

Example:

The user should be able to upload a PDF.

💻 Practical Task 3

Write at least **5 non-functional requirements**.

Example:

The system should respond to user requests within a reasonable time.

💻 Practical Task 4

Create at least **3 use cases** for your application.

💻 Practical Task 5

Define:

- Project Scope
- Constraints
- Assumptions
- Expected Outcome